D1 Reading tracebacks and stack traces

Find the root cause in Python and Spark errors.

D1-L1 Read a Python traceback from the bottom up

Read the last line, then the frame above it, then walk up to your own code. When you re-raise, add context and chain with `from err`.

```
Traceback (most recent call last):
  File "observability.py", line 7, in <module>
    main()
    ~~~~^^
  File "observability.py", line 5, in main
    log_run()
    ~~~~~~~^^
  File "observability.py", line 2, in log_run
    return str(uuid.uuid4())
               ^^^^
NameError: name 'uuid' is not defined. Did you forget to import 'uuid'?
```

Order for reading a traceback	Last line (what), frame above it (where), then up to the first frame in your own code.
'The above exception was the direct cause...'	Someone chained with raise ... from err. The first traceback is the root cause.
'During handling of the above exception...'	The except block raised a new error. Fix the handler and the original error.
Re-raising with context	Catch the narrow exception, add facts to the message, and chain with from err.

D1-L2 Spark errors, the condition and the real cause

Read the condition, the SQLSTATE class and the origin line. Class 42 means fix the code. Class 22 means look at the data.

```
[UNRESOLVED_COLUMN.WITH_SUGGESTION] A column, variable, or function
parameter with name `final_wieght` cannot be resolved. Did you mean one
of the following? [`final_weight`, `age`, `wave_id`, `respondent_id`].
SQLSTATE: 42703
```

Shape of a Spark 4 error message	[ERROR_CONDITION] message ... SQLSTATE: xxxxx, often with an origin line under == DataFrame ==.
SQLSTATE class 42 vs 22	42: syntax error or access rule violation, fix the code. 22: data exception, look at the data.
Why a data error appears at write()	Transformations are lazy. The action runs the plan. The origin line shows where the expression was written.
NOT_COLUMN_OR_STR ... got generator	Python's sum is shadowed by pyspark.sql.functions.sum. Import functions as F.

D1-L3 Py4J and Spark Connect errors

Find the last "Caused by" in a Py4J error. Under Spark Connect, expect errors at the action, and catch pyspark.errors classes.

```
Py4JJavaError: An error occurred while calling
z:org.apache.spark.api.python.PythonRDD.collectAndServe.
: org.apache.spark.SparkException: Job aborted due to stage failure ...
Caused by: org.apache.spark.api.python.PythonException: Traceback ...
  File "/repo/job.py", line 5, in <lambda>
```

```
6 ZeroDivisionError: division by zero
```

Finding the cause in a Py4JJavaError	Search for the last 'Caused by:'. A Python UDF error holds a Python traceback there.
Spark Connect and missing columns	Analysis is deferred. The error comes at the action or when you read df.columns or df.schema.
Exception class that works on Classic and Connect	pyspark.errors.AnalysisException (and other pyspark.errors classes). Not Py4JJavaError.
Serverless compute and the Spark API	Spark Connect only. No RDD APIs. spark.sparkContext raises JVM_ATTRIBUTE_NOT_SUPPORTED.